

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

### CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

|                |                                                                                                                                                 |               |                                                         |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------------------------------------------------------|
| Course Code:   | CSA10 / CSA1063                                                                                                                                 | Course Title: | Software Engineering                                    |
| CO Assessed:   | CO2                                                                                                                                             | Assessment:   | Assessment Tool 2 – Scenario-Based Requirement Analysis |
| Weightage:     | 20%                                                                                                                                             | Total Marks:  | 25                                                      |
| CO2 Statement: | <i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i> |               |                                                         |
| Student Name:  | N Goutham                                                                                                                                       | Reg. No.:     | 192472024                                               |
| Date:          | 06-08-2026                                                                                                                                      | Duration:     | 60 minutes                                              |

#### Code of Conduct

*I N Goutham/192472024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: N Goutham

#### Annexure A – Scenario-Based Requirement Analysis

##### Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

##### Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

- 1. Study and Analyze the Scenario**
  - Carefully understand the given problem statement.
  - Identify the business domain, stakeholders, existing challenges, and system objectives.
- 2. Identify Functional and Non-Functional Requirements**
  - List all functional requirements required by the proposed system.
  - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
- 3. Identify Actors and Use Cases**
  - Identify all primary and secondary actors interacting with the system.
  - List the major use cases associated with each actor.
  - Draw a Use Case Diagram representing the interactions between actors and the system.
- 4. Prepare the Software Requirement Specification (SRS)**

Develop an SRS document containing:

  - Introduction
  - Problem Description
  - Scope of the System

- Functional Requirements
  - Non-Functional Requirements
  - Use Case Descriptions
  - Data Requirements
  - External Interface Requirements
  - System Features
5. **Identify Assumptions and Constraints**
    - List the assumptions considered while developing the system.
    - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
  6. **Validate Requirement Completeness and Consistency**
    - Verify whether all stakeholder requirements have been addressed.
    - Check for ambiguity, redundancy, inconsistency, and missing requirements.
    - Suggest improvements to enhance the quality and completeness of the requirement specification.

# **1. SCENARIO UNDERSTANDING AND PROBLEM ANALYSIS**

## **1.1 Project Title**

### **Open-Source Contribution Tracker and Recommendation System for Beginner Developers**

## **1.2 Introduction**

Open-source software has become an important part of modern software development. Platforms such as GitHub provide access to a large number of software projects where developers can collaborate, solve problems, improve existing applications, and gain practical programming experience. For beginner developers, contributing to open-source projects is a valuable way to develop technical skills, understand real-world software development practices, and build a professional portfolio.

However, entering the open-source ecosystem can be difficult for beginners. GitHub contains millions of repositories and a very large number of issues. A beginner may know a programming language such as Python, Java, JavaScript, or C++, but may not know which project or issue is suitable for their current skill level. Searching manually for appropriate issues can consume significant time.

The proposed Open-Source Contribution Tracker and Recommendation System aims to solve this problem by providing a centralized platform that recommends suitable beginner-friendly issues according to a user's programming skills and interests. The system can retrieve repository and issue information through the GitHub API and identify issues containing labels such as "good first issue" and "help wanted."

The system will also track the user's contributions across different repositories. It will provide information about issues, pull requests, merge status, contribution frequency, technology diversity, and skill progression. This allows beginners to understand their growth and create a stronger open-source portfolio.

## **1.3 Existing Problem**

Currently, beginners generally depend on manual GitHub searches, online tutorials, community recommendations, or social media to identify suitable projects. This approach does not provide a centralized method for matching a developer's skills with appropriate issues.

A beginner may encounter an issue that appears simple but requires advanced knowledge of a particular framework or technology. Similarly, a developer may miss suitable issues because they are distributed across many repositories.

Another major problem is contribution tracking. When a developer contributes to multiple repositories, their issues and pull requests are distributed across different projects. Maintaining a complete contribution history manually becomes difficult.

Therefore, there is a need for a system that combines recommendation, contribution tracking, progress monitoring, notifications, and portfolio generation in a single platform.

## **2. OBJECTIVES, STAKEHOLDERS AND SCOPE**

### **2.1 Objectives**

The main objectives of the proposed system are:

1. To recommend beginner-friendly open-source issues based on the user's programming skills and interests.
2. To integrate with GitHub for retrieving repository, issue, and pull request information.
3. To filter issues using beginner-friendly labels such as "good first issue" and "help wanted."
4. To track contributions made by a developer across multiple repositories.
5. To provide a dashboard showing contribution history and progress.
6. To estimate the difficulty of repositories and issues.

### **2.2 Stakeholders**

#### **Beginner Developer**

The beginner developer is the primary stakeholder and main user. The user provides programming skills and interests and receives personalized issue recommendations. The user can also track contributions, monitor pull requests, view progress, and generate a portfolio.

#### **Open-Source Project Maintainer**

Project maintainers manage repositories and issues. Their project activity, issue labels, response patterns, and contribution information can help the system provide community insights to users.

#### **GitHub**

GitHub acts as an external platform that provides repository, issue, pull request, and contribution-related information through its API.

#### **System Administrator**

The administrator is responsible for monitoring the application, managing system-level operations, handling errors, and maintaining the system.

## **Open-Source Community**

The open-source community includes contributors, maintainers, mentors, and other developers who participate in projects. Their activities influence the contributor-friendliness of projects.

### **2.3 Scope of the System**

The system will provide a web-based platform where developers can create a profile containing their programming languages, technologies, interests, and skill levels.

The system will connect with GitHub and retrieve relevant repositories and issues. It will identify beginner-friendly issues and recommend them based on the user's profile.

The system will also provide contribution tracking and a skill progression dashboard. Users will be able to monitor their issues, pull requests, merge status, contribution frequency, and technology diversity.

## **3. FUNCTIONAL REQUIREMENTS**

Functional requirements describe the services and operations that the system must provide.

### **FR1 – User Registration and Login**

The system shall allow a new developer to create an account using the required registration details. Registered users shall be able to securely log into the system.

### **FR2 – Developer Profile Management**

The system shall allow users to create and update their developer profile. The profile shall contain relevant information such as programming languages, technologies, interests, and skill level.

### **FR3 – Skill and Interest Selection**

The system shall allow users to select the programming languages and technologies they know and specify their areas of interest. This information shall be used by the recommendation engine.

### **FR4 – GitHub Integration**

The system shall allow users to connect their GitHub account. The system shall use the GitHub API to retrieve relevant repository, issue, pull request, and contribution information.

### **FR5 – Repository Retrieval**

The system shall retrieve open-source repository information from GitHub. Repository information may include repository name, programming language, activity, issues, and other publicly available information.

## **4. NON-FUNCTIONAL REQUIREMENTS AND DATA REQUIREMENTS**

### **4.1 Non-Functional Requirements**

Non-functional requirements describe the quality and operational characteristics of the proposed system.

#### **4.1.1 Performance**

The system should provide search and recommendation results within a reasonable amount of time. API requests should be processed efficiently so that users do not experience unnecessary delays.

#### **4.1.2 Security**

User authentication information must be protected from unauthorized access. GitHub authentication tokens, if used, must be securely managed. Administrative functions must only be accessible to authorized users.

#### **4.1.3 Usability**

The system should have a simple and beginner-friendly interface. Users should be able to understand recommended issues, difficulty scores, and contribution statistics without requiring advanced technical knowledge.

#### **4.1.4 Reliability**

The system should continue to operate reliably even when temporary errors occur while communicating with GitHub. Appropriate error messages should be displayed when required information cannot be retrieved.

### **4.2 Data Requirements**

The system needs to maintain several categories of data.

#### **User Data**

- User ID
- Name
- Email
- Authentication information
- GitHub username

#### **Skill Data**

- Programming languages
- Technologies
- Skill level
- Areas of interest

#### **Repository Data**

- Repository name
- Repository owner
- Programming language
- Repository URL
- Activity information
- Difficulty score

### **Issue Data**

- Issue ID
- Issue title
- Issue description
- Labels
- Repository
- Issue status
- Difficulty score

The stored data will allow the system to provide personalized recommendations and maintain a history of the user's contribution journey.

## **5. ACTORS AND USE CASES**

### **5.1 Actors**

The proposed system contains one primary actor and several supporting actors.

#### **Primary Actor – Beginner Developer**

The beginner developer interacts directly with the system. The developer can register, manage a profile, provide skills and interests, connect GitHub, search for issues, receive recommendations, track contributions, monitor progress, receive notifications, and generate a portfolio.

#### **Secondary Actor – GitHub API**

The GitHub API provides repository, issue, pull request, and contribution information required by the system.

#### **Secondary Actor – System Administrator**

The administrator manages and monitors the system and handles system-level problems.

#### **Secondary Actor – Notification Service**

The notification service delivers alerts to users about relevant issues and updates.

#### **Secondary Actor – Project Maintainer**

Project maintainers indirectly interact with the system through repository and issue information. Their project activity can contribute to community insight calculations.

## 5.2 Major Use Cases

| Use Case                 | Actor              |
|--------------------------|--------------------|
| Register/Login           | Beginner Developer |
| Manage Profile           | Beginner Developer |
| Add Skills and Interests | Beginner Developer |
| Connect GitHub           | Beginner Developer |
| Search Repository        | Beginner Developer |
| Search Issues            | Beginner Developer |
| Get Recommendations      | Beginner Developer |
| View Difficulty Score    | Beginner Developer |

## 5.3 Use Case Description – Issue Recommendation

**Actor:** Beginner Developer

**Supporting Actor:** GitHub API

**Precondition:** The user has created a profile containing programming skills and interests.

### Main Flow:

1. The user opens the recommendation section.
2. The system retrieves the user's skill profile.
3. The system requests suitable issue information from GitHub.
4. The system identifies beginner-friendly issues.
5. The system evaluates the compatibility between the user's skills and available issues.
6. The system calculates or retrieves difficulty information.
7. The system ranks the issues.
8. The recommended issues are displayed to the user.

### Postcondition:

The user receives a list of open-source issues suitable for their current skill profile.

## 5.4 Use Case Description – Contribution Tracking

**Actor:** Beginner Developer

**Supporting Actor:** GitHub API

1. The user connects their GitHub account.
2. The system retrieves contribution information.
3. The system identifies relevant issues and pull requests.
4. The system records the contribution information.
5. The dashboard is updated.
6. The user can view their contribution history.

# 6. SYSTEM FEATURES AND EXTERNAL INTERFACES

## 6.1 Personalized Recommendation Engine

The recommendation engine is one of the main features of the proposed system. It uses information provided by the developer, such as programming languages, technologies, interests, and skill level, to identify relevant open-source issues.

The system retrieves suitable issues from GitHub and filters them based on beginner-friendly characteristics. The issues are then ranked according to their relevance to the user's profile.

## **6.2 GitHub Integration**

The system communicates with GitHub through its API. This integration allows the application to retrieve repository information, issue information, labels, pull requests, and other relevant public information.

The GitHub integration reduces the need for users to manually enter their contribution information.

## **6.3 Contribution Tracking**

The contribution tracker provides a centralized view of the user's activities across different repositories. It allows users to see their issues, pull requests, merge status, and contribution history in one place.

## **6.4 Skill Progression Dashboard**

The dashboard provides a visual representation of the user's progress. The system can consider the number of contributions, technologies used, and complexity of completed contributions.

This helps beginners understand how their practical development experience is increasing over time.

## **6.5 External Interface Requirements**

### **User Interface**

The system shall provide interfaces for:

- Login and registration
- Developer profile
- Skill selection
- Repository search
- Issue recommendations
- Contribution tracking
- Skill progression
- Notifications
- Portfolio generation
- Community insights

### **GitHub API Interface**

The system shall communicate with GitHub to retrieve required repository, issue, pull request, and contribution information.

### **Notification Interface**



The notification component shall communicate with the application to deliver relevant alerts to users.

## 6.9 Use Case Diagram

**Insert the UML Use Case Diagram here.**

The diagram should contain:

### **Beginner Developer →**

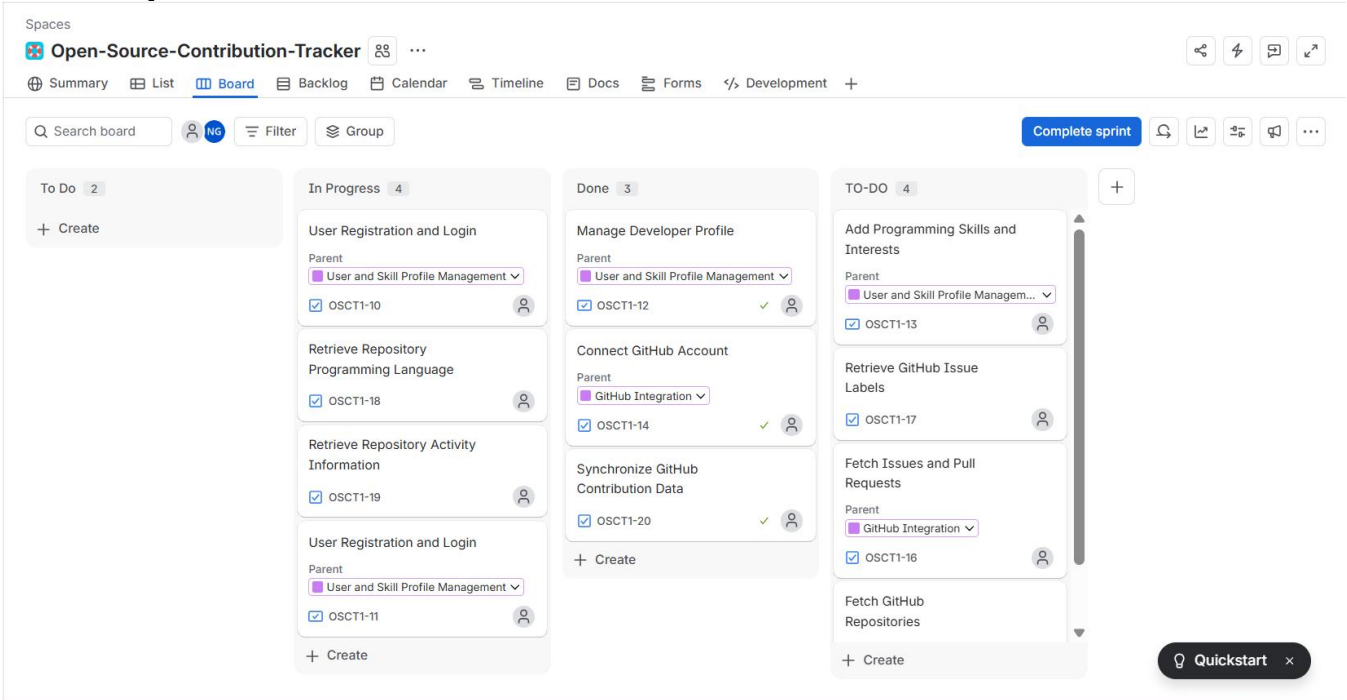
- Register/Login
- Manage Profile
- Manage Skills
- Connect GitHub
- Search Issues
- Get Recommendations
- View Difficulty
- Track Contributions
- Track Pull Requests
- View Skill Progress
- Receive Notifications
- Generate Portfolio
- View Community Insights

### **GitHub API →**

- Repository Data
- Issue Data
- Pull Request Data
- Contribution Data

This diagram is important because the assessment specifically requires a UML Use Case Diagram showing actor-system interactions.

# Tool Output Screenshot



## Rubrics for Calculation

| Criteria                                                              | Excellent (4 Marks)                                                                                                        | Good (3 Marks)                                                  | Satisfactory (2 Marks)                                          | Needs Improvement (1 Mark)                                           |
|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------|
| <b>Scenario Understanding and Problem Analysis (30%)</b>              | Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.   | Demonstrates good understanding with minor omissions.           | Basic understanding with limited analysis.                      | Poor understanding; major aspects missing or incorrect.              |
| <b>Functional and Non-Functional Requirement Identification (25%)</b> | Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.                | Identifies most requirements with minor classification errors.  | Identifies only basic requirements; several omissions.          | Requirements are incomplete, inaccurate, or poorly classified.       |
| <b>Actor Identification and Use Case Modelling (20%)</b>              | Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards. | Most actors and use cases identified; diagram has minor errors. | Limited actors/use cases identified; diagram partially correct. | Actors/use cases missing or incorrect; diagram absent or inaccurate. |

|                                                                     |                                                                                                                                                               |                                                                      |                                                             |                                                                     |
|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Software Requirement Specification (SRS) Documentation (15%)</b> | SRS is well-structured, complete, professionally organized, and includes all required sections.                                                               | SRS is organized with minor missing sections or formatting issues.   | SRS includes only essential sections with limited detail.   | SRS is incomplete, poorly organized, or lacks essential components. |
| <b>Assumptions, Constraints, and Requirement Validation (10%)</b>   | Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification. | Identifies most assumptions and constraints; validation is adequate. | Limited assumptions/constraints; validation is superficial. | Assumptions, constraints, and validation are missing or incorrect.  |

| Criteria                                                       | Mark       |
|----------------------------------------------------------------|------------|
| Scenario Understanding and Problem Analysis (30%)              | /5         |
| Functional and Non-Functional Requirement Identification (25%) | /5         |
| Actor Identification and Use Case Modelling (20%)              | /5         |
| Software Requirement Specification (SRS) Documentation (15%)   | /5         |
| Assumptions, Constraints, and Requirement Validation (10%)     | /5         |
| <b>TOTAL</b>                                                   | <b>/25</b> |